Waveguides and Antennas [EE3140]

Programming Assignment - 1

EE24BTECH11201 Shiva Varun Adicherla

A Part

A.1

The spatial division used in the simulation is given by:

$$\Delta z = \frac{\text{Domain Size}}{N-1} \Rightarrow \frac{6m}{2000} \approx 0.3 \text{ cms}$$

This is used because between N points we'll have "N-1" Δz, which we are assuming will span the whole domain. The **Odd** number of points were chosen because it allows for a symmetric division of the domain into 2 equal halves.

The simulation time step I've used is $\boldsymbol{dt = 5 \times 10^{-13} s}$, along with simulating for **15ns**, because, if we look at the source, it's basically a guassian so we start with the exponential at $-\left(\frac{5}{2}\right)^2$ and we'll land back to the same at $\boldsymbol{t = 10\text{ns}}$, so dividing the time I want to simulate with the time step yields:

$$\text{Number of Steps} = \frac{T}{dt} \Rightarrow 3 \times 10^4$$

Now for the spatial resolution, which I've chosen to interpret in the following way:

$$\text{Spatial Resolution} = \frac{\lambda}{\Delta z}$$

λ isn't hard to calculate and is given by $\lambda = c_0 \times f^{-1} \Rightarrow 30$ cms, and out spatial resolution will be given by:

$$\text{Spatial Resolution} = \frac{30}{0.3} \approx 100$$

The courant condition from my understanding is essentially a part of establishing **causality** in the simualtion, and we achieve this by making sure that the information flow only happens to the adjacent cells/points.

$$\max(V_{\text{in the simulation}}) \times dt \leq \Delta z \Longrightarrow dt \leq \frac{\Delta z}{\max(V_{\text{in the sim}})}$$

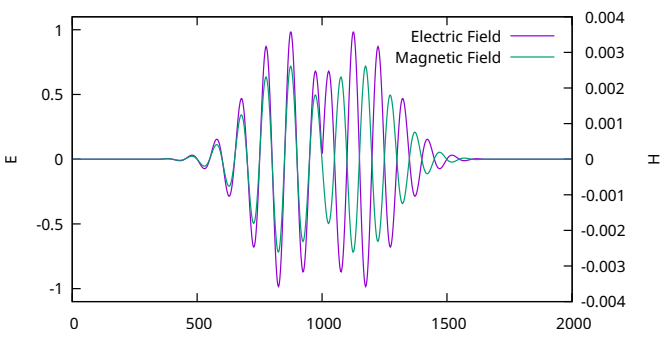
Since this is free space, we can approximate the speed of light $c_0 \approx 3 \times 10^8$ m/s, so our dt's limit will be:

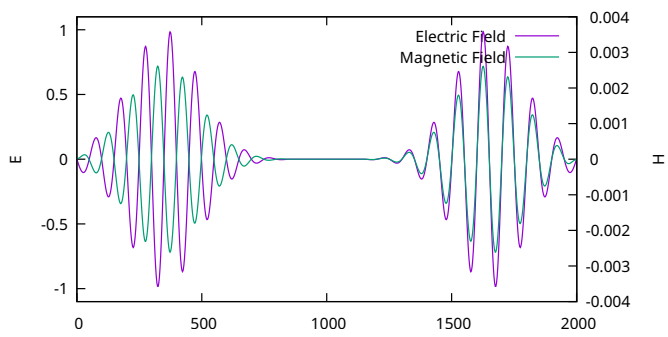
$$dt \leq \frac{3 \times 10^{-3}}{3 \times 10^8} \Rightarrow dt \leq 10\text{ps}$$

Which I've considered and satisfied, by taking $dt = 5 \times 10^{-13}$.

A.2

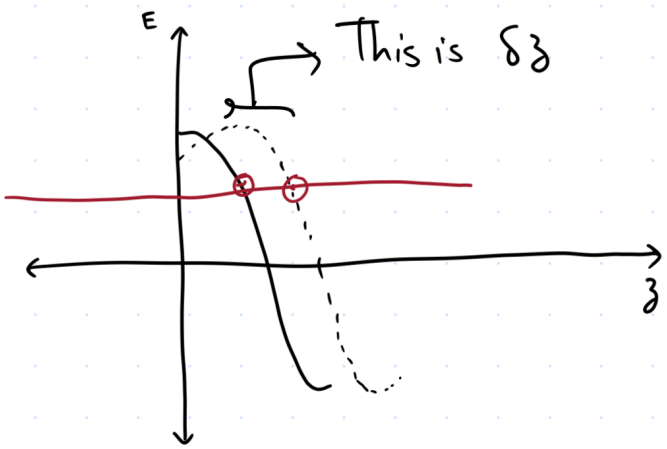
I've animated the evolution of electric and magnetic fields in gnuplot and the gifs and the source code are in my Github repo, but some of the images are attached below.





A.3

To estimate the propagation velocity, I've essentially fixed a level whose position I'm going to check in the next instant. Illustrated by the following figure.



The red line represents the level I fixed, the solid line the current E, dashed the future E

A simple warning though, this approach requires us to either find the exact values of the intersections which we can guarentee by **Linearly Interpolating** between the points in which we expect the intersection to be. And this too only is **accurate at the zero crossing points** because the slope is maximum there. Having a low slope at the point of interest will cause issues with numerical precision because the target value is already close.

The velocity then is given by the following:

$$V = \frac{\delta z}{dt}$$

```
  double target = g->Egrid[i];
                ... // This is not the complete code, just an outline
    double e1 = g->Egrid_prev[j];
    double e2 = g->Egrid_prev[j + 1];

    // Find the region in which the target lives
    if ((e1 <= target && target <= e2) || (e2 <= target && target <= e1)) {
      double alpha = 0.0;
      double diff = e2 - e1;
      if (fabs(diff) > 1e-13) { // Failsafe if Slope is really small
        alpha = (target - e1) / diff;
      }
                ...
      }
    }
  }
                ...
  double dz = ((double)i - alpha) * g->dx;
  return dz / dt;
}
```

After choosing a proper point, and using this method, I've calculated the velocity to be:

$$V \Rightarrow c_0 = 299757617.346131 \text{ m/s } (\delta = 0.0116\%)$$

Determining the Intrinsic Impedance is straight forward and can be evaluated anywhere except for the points at which H is very low, and lead to numerical errors, it is given by:

$$\eta = \frac{E}{H}$$

```
printf("Calculated Impedance is :%lf Ohm\n",
        fabs(2 * g.Egrid[1620] / (g.Hgrid[1620] + g.Hgrid[1619])));
```

Since we have the H and E field's grid staggered according to the Yee's Method, we'll have to account for this by avergaing out the values of H adjacent to the E grid point, Which I'm going in the code above.

Here, I've chosen a point where the Electric and Magnetic fields were at the maximum, to avoid the said numerical error.

$$\eta = \frac{E}{H} \Rightarrow 376.7605\ \Omega \quad (\delta = 0.605\%)$$

A.4

The electric field undergoes throught the polarity inversion because of the boundary condtions imposed by the maxwell's equations, which essentially state that the tangential electric field at the surface of PEC boundary must be 0, so the reflected Electric field is forced to have a polarity opposite to that of the incident Electric Field. As a result of this, we'll also have some surface current on the PEC boundary which also forces the Magnetic field inside the PEC to be 0 too.

Program output

```
❯ ls
- makefile  parta  parta.c  plot1.pdf  plot2.pdf  plot.gif
❯ ./parta
- calculated velocity is :299757617.346131 m/s
- calculated impedance is :376.760500 ohm
```

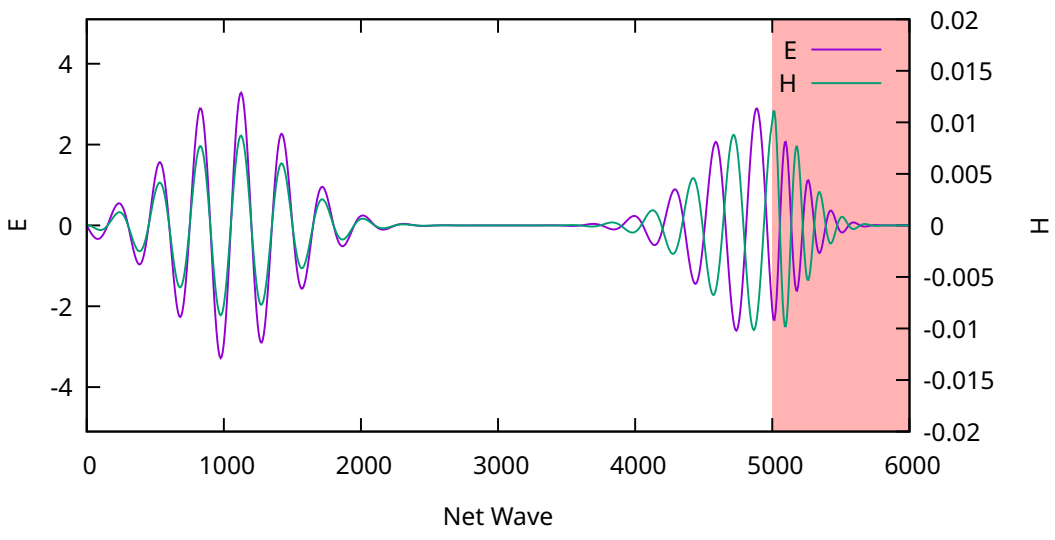
B Part

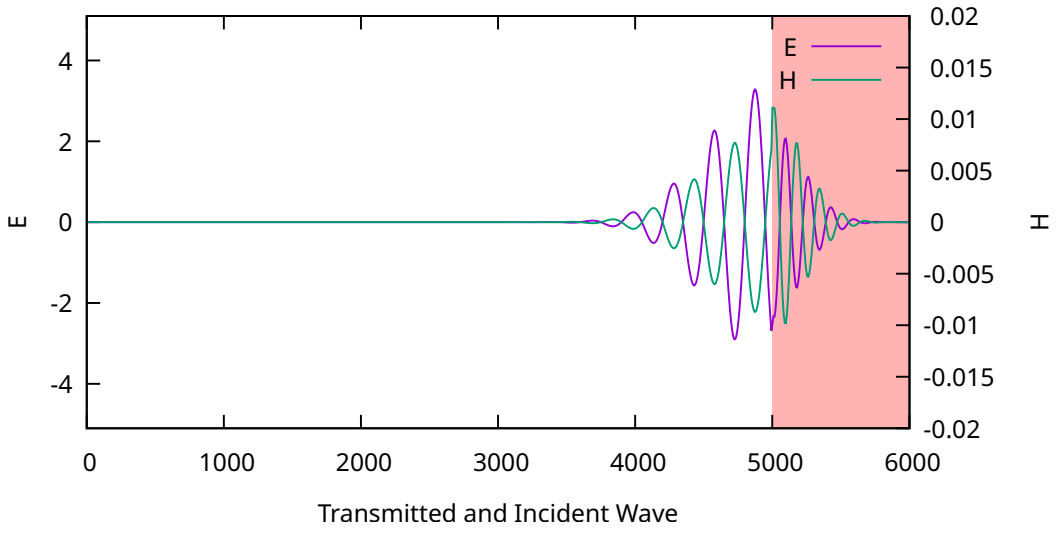
For this part of the question, to avoid additional reflections from the source (The source I've used in the previous part is a hard source), I've changed the source type here to a **soft source**, since there isn't a constraint mentioned in the question sheet.

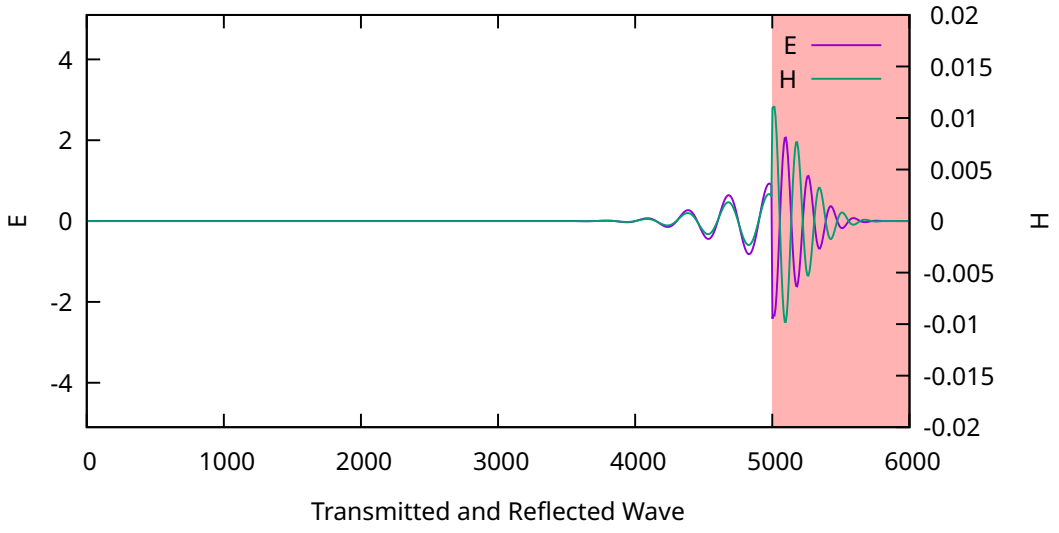
B.1

Here the identification of Incident and Transimitted waves is straight forward, but the identification of reflected wave is not as straight forward. As given in the note of the question, we'll have the reference incident wave from the simulation we have had in Part A or the wave that was generated in the other direction as the reference wave in the free space. Subtracting the complete wave we have in this simulation with the reference wave we had, will essentially yeild **us the pure reflected wave for a brief Interval.**

The pictures are attached below:







The figure labelled net wave is the RAW simulation data, and other 2 figures contain the transmitted wave in the RED coloured region (Basically marks the Dielectric with $\varepsilon_r = \mathbf{3.2}$) and the Reflected/Transmitted part in the White/Nocolor Region according to the figure label.

```
if (i == 23000) { // Timestep at which I'm capturing the snapshot
 double tempE[g.N];
 double tempH[g.N];
 // Reflected Wave
 for (int i = 1; i < 2001; i++) {
   // E field is flipped since I'm using the wave that was
   // propagating in the other direction.
   tempE[3000 + i] = g.Egrid[3000 + i] - g.Egrid[3000 - i];
   tempH[3000 + i] = g.Hgrid[3000 + i] + g.Hgrid[3000 - i];
 }
 for (int i = 5001; i < 6001; i++) {
   tempE[i] = g.Egrid[i];
   tempH[i] = g.Hgrid[i];
 }
 gplotSetOption(&g1, "set output \"SeperateWave-TR.pdf\"");
 gplotSetOption(&g1, "set xlabel \"Transmitted and Reflected Wave\"");
 gplotPBDSEM(&g1, tempE, tempH, "E", "H", g.N);
 // Incident wave
 for (int i = 3001; i < 5000; i++) {
   tempE[i] = g.Egrid[6000 - i];
   tempH[i] = -g.Hgrid[6000 - i];
 }
```

```
    gplotSetOption(&g1, "set output \"SeperateWave-TI.pdf\"");
    gplotSetOption(&g1, "set xlabel \"Transmitted and Incident Wave\"");
    gplotPBDSEM(&g1, tempE, tempH, "E", "H", g.N);
    // Complete wave
    gplotSetOption(&g1, "set output \"SeperateWave-NW.pdf\"");
    gplotSetOption(&g1, "set xlabel \"Net Wave\"");
    gplotPBDSEM(&g1, g.Egrid, g.Hgrid, "E", "H", g.N);
}
```

B.2

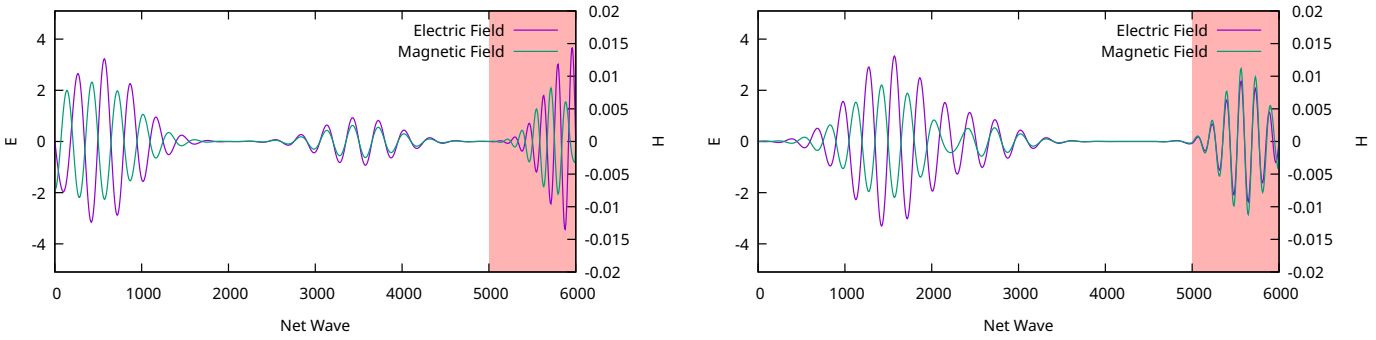
Similar to the question in Part A, we can extract the value of the velocity using the same method, which yeilds us:

$$V = 167516463.117863 \text{ m/s } (\delta = 0.04\%)$$

$$\text{We can verify this with } V = \frac{1}{\sqrt{\varepsilon\mu}} \Rightarrow \frac{c_0}{\sqrt{3.2}} = 167589078.80743 \text{ m/s}$$

B.3

For calculating the Transmission and Reflection co-efficients, I've basically taken time instants at which the target part **of the wave is isolated**, then identified the peak/Extrema value of Electric field over the target range which I've found out by inspection. I've chosen the time instants depicted in the following graphs/figures.



The figure on the left is/was used to extract the peak value of Electric Field in Incident wave and Reflected Wave, where as the figure in the right was used to extract the peak Value of Electric Field in the dielectric. The main idea is that the **peak stays at the same spot with respect to the whole wave**. Then dividing the Electric Field Amplitudes will yeild us the required Co-efficients as they are defined as such.

$$\Gamma = \frac{E_{\text{Reflected}}}{E_{\text{Incident}}} \Rightarrow \frac{\eta_2 - \eta_1}{\eta_2 + \eta_1} \qquad \text{T} = \frac{E_{\text{Transmitted}}}{E_{\text{Incident}}} \Rightarrow \frac{2\eta_2}{\eta_2 + \eta_1}$$

Calculating the values of Γ and T from the given information, we get:

$$\Gamma = \left(\frac{\frac{1}{\sqrt{3.2}} - 1}{\frac{1}{\sqrt{3.2}} + 1}\right) \Rightarrow -0.28286 \qquad \text{T} = \left(\frac{\frac{2}{\sqrt{3.2}}}{\frac{1}{\sqrt{3.2}} + 1}\right) \Rightarrow 0.71714$$

Extracted values of Γ and T from field amplitude samples are:

$$\Gamma = -0.282924 \ (\delta = 0.022\%) \qquad \text{T} = 0.716769 \ (\delta = 0.051\%)$$

The sign extraction of the co-efficients was possible because we had asymetric waveform with regards to the maximum/minimum values. I've exploited this fact to extract the sign, and the code responsible and it's explaination is given in the next page.

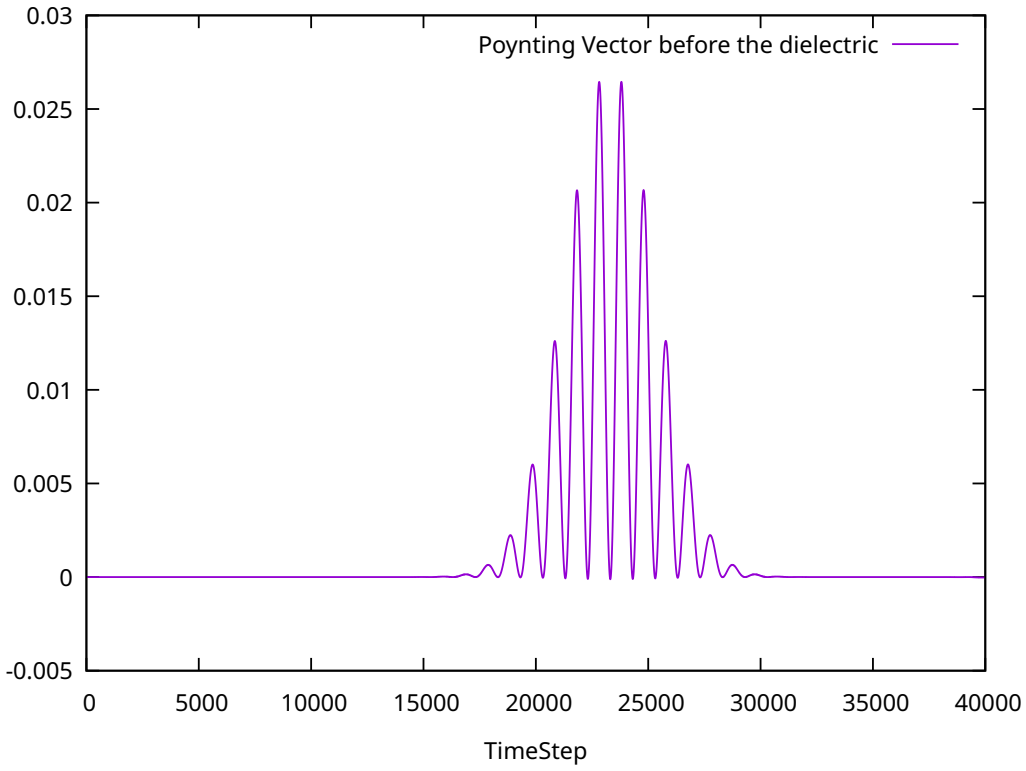
(Continued in the next page)

```
if (i == 16000) // TimeStep at which I'm simpling the Peaks
    incidentMax = (findMax(g.Egrid, g.Egrid + g.N - 1) >
                   fabs(findMin(g.Egrid, g.Egrid + g.N - 1))) // If the +ve peak is greater
                      ? findMax(g.Egrid, g.Egrid + g.N - 1) // Take the amplitude as +ve Max
                      : findMin(g.Egrid, g.Egrid + g.N - 1); // Else we take the -ve Peak
if (i == 33333) {
  printf("The velocity of the EM Wave in the Dielectric is: %lf\n",
         returnVelocity(&g, 5020, dt, 2));
  reflectedMax = (findMax(g.Egrid + 3000, g.Egrid + 4000) >
                  fabs(findMin(g.Egrid + 3000, g.Egrid + 4000)))
                     ? findMax(g.Egrid + 3000, g.Egrid + 4000)
                     : findMin(g.Egrid + 3000, g.Egrid + 4000);
}
if (i == 39999) {
  transmittedMax = (findMax(g.Egrid + 5001, g.Egrid + 6000) >
                    fabs(findMin(g.Egrid + 5001, g.Egrid + 6000)))
                       ? findMax(g.Egrid + 5001, g.Egrid + 6000)
                       : findMin(g.Egrid + 5001, g.Egrid + 6000);
}
```

B.4

Assumption: Since the question doesn't mention if the poynting vector plot should be with respect to the time or with respect to the z, I'm going to consider the plot to be along time.

The calculation of poynting vector is straight forward, with us **just taking $\boldsymbol{E} \times \boldsymbol{H}$** and since E is along $\vec{j}$ and H along $\vec{x}$, we'll have to compensate with a **-ve sign.** (Since $\vec{j} \times \vec{i} = -\vec{k}$)



Identification of power flow inside the simulation is easy - we can just look at the sign of poynting vector, If the sign is +ve, the power flow is towards $+\vec{z}$ and when it's -ve it's towards $-\vec{z}$.
We can prove/find out the average power flow through integrating the poynting flux at a point, which we can give by just doing the following:

$$\sum_{\tau} E_k[\tau] = \sum_{\tau} E_l[\tau] \qquad \text{Where } k \neq l$$

```
// Book-keeping for Part 4
poyntingVec1[i] = -(g.Egrid[4996]) * (g.Hgrid[4996] + g.Hgrid[4997]) / 2;
// printf("%lf\n", poyntingVec1[i]);
poyntingInt1 -= (g.Egrid[4996]) * (g.Hgrid[4996] + g.Hgrid[4997]) / 2;
poyntingInt2 -= (g.Egrid[5004]) * (g.Hgrid[5004] + g.Hgrid[5005]) / 2;
```

So essentially, if the values of Time integral of poynting vector at 2 (different) points is the same, we'll have proved that Energy conservation indeed holds true at the interface between free-space and dielectric.

Program output

```
❯ ls
- Makefile  partB  partB.c  plot1B.pdf  plot2B.pdf  plot.gif  PoyntingVec.pdf  SeperateWave-NW.pdf
SeperateWave.pdf  SeperateWave-TI.pdf  SeperateWave-TR.pdf
❯ ./partB
- The velocity of the EM Wave in the Dielectric is: 167516463.117863
- Reflection Co-efficient is: -0.282924
- Transmission Co-efficient is: 0.716769
- Time Integral of Poynting Vector before and after the dielectric boundary: 68.069187 68.044646
```

And we can indeed see that the **time average of poynting vector basically yeilded the approximately the same values, hence confirming that the interface is lossless.**

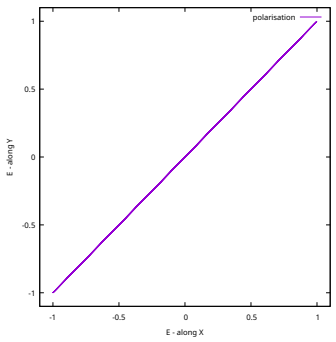
C Part

I really don't need to extend the simulation framework, since we have projected the wave into orthogonal basis, so I can instead run 2 simulations independently (Maxwell's equations are essentially linear here) and combine the results.

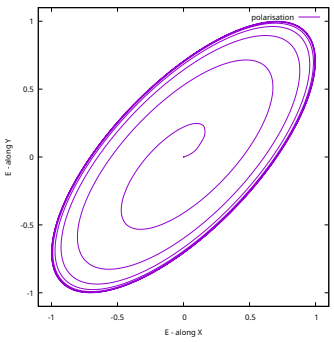
C.1

I've chosen an observation point around $z = 0.6m$, and used 2 1D simulations, and combined the results to get the resulting polarisation trajectory. Since we can't have anything that's interactive in this pdf/report, I'm omitting the time evolution of the polarisation graph. It is however available on my git repository, that's given at the end of this report.

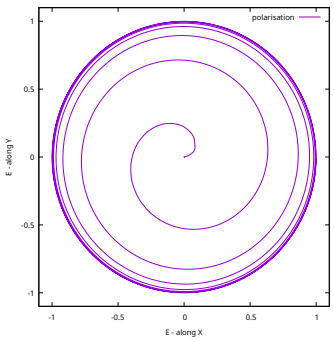
C.2



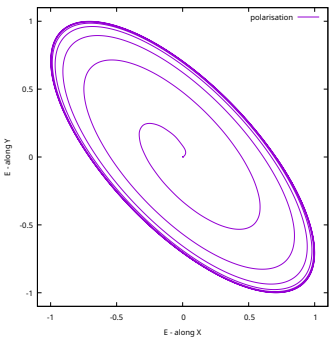
$\varphi = 0$ rad



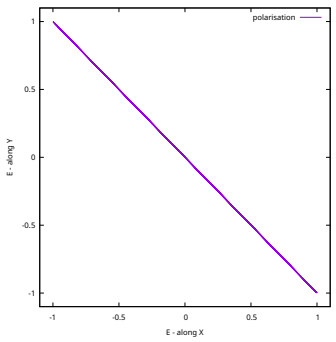
$\varphi = \frac{\pi}{4}$ rad



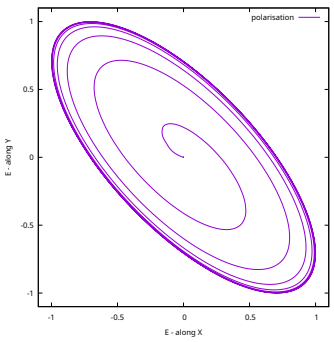
$\varphi = \frac{\pi}{2}$ rad



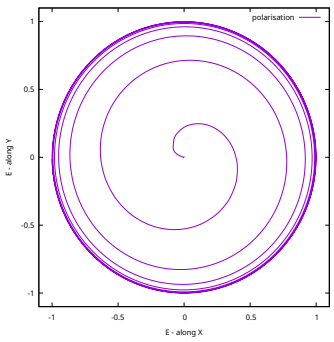
$\varphi = \frac{3\pi}{4}$ rad



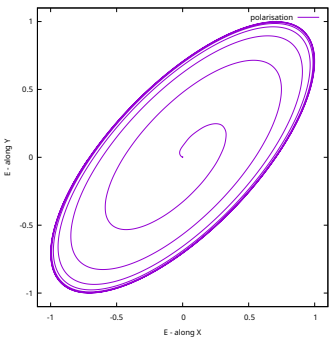
$\varphi = \pi$ rad



$\varphi = \frac{5\pi}{4}$ rad



$\varphi = \frac{3\pi}{2}$ rad



$\varphi = \frac{7\pi}{4}$ rad

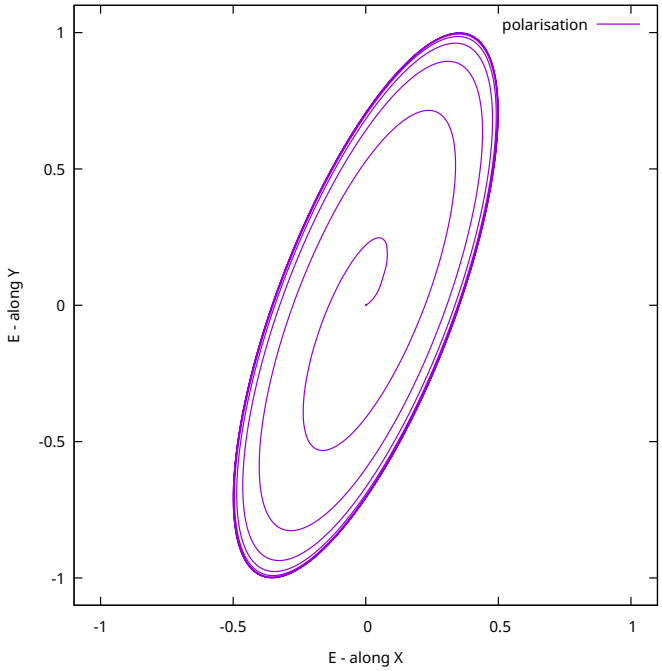
I obtained the above figures after sweeping the φ over $[0, 2\pi)$ in steps of $\frac{\pi}{4}$. The perspective of the above plots is of the view when the wave is moving out of the paper/screen.

C.3

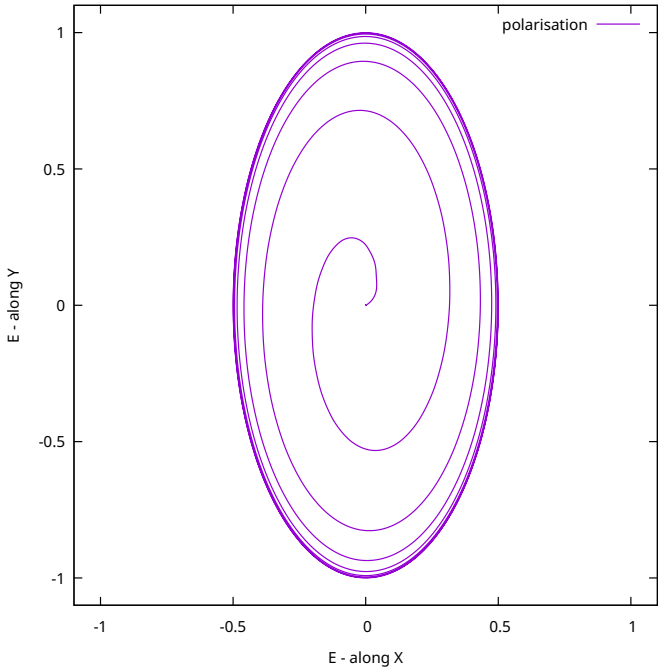
As discussed in the class, we can describe this with the following equation.

$$\left(\frac{x}{E_x}\right)^2 + \left(\frac{y}{E_y}\right)^2 - \frac{2xy}{E_x E_y} = \sin^2(\varphi)$$

Below are the figures which vary both φ and Amplitudes of the waves.



$\varphi = \frac{\pi}{4}$ rad



$\varphi = \frac{\pi}{2}$ rad

The convention used to describe the direction of polarisation is also discussed in the class and is given by the hand which is able to twist in the direction of rotation polarisation while the thumb of the said hand is in the direction of propagation of the EM wave.

GitHub Repository

The github repository can be accessed here. It will include a readme for this Programming assignment holding all the animated Gifs, and the requested source code.

END